

Coding and Methodology for Image Analysis of Wood Pyrolysis

By Ian Zalewski - iaz228@lehigh.edu

For Dr. Andrea Dernbecher - andrea.dernbecher@tu-dortmund.de

Abstract:

This report covers the topic of wood pyrolysis and a program that uses OpenCV2 image analysis to capture the dimensions of a wood particle as it undergoes pyrolysis. This document will also have instructions for use of the program, recommended settings, and further improvements that can be made.

Methodology:

Data gathering comes from an experiment run on wood particles put into an oxygenless reactor. This chamber is then heated over time for the wood to undergo pyrolysis. This chamber has multiple windows that view the particle. In two of these windows, cameras are set up. One is a high speed silhouette camera. The other is a phosphor camera for reading the temperature. These cameras pick up separate angles of the particle. The particle usually undergoes shrinking and cracking during the heating. This size change over time is of interest.

To pick up the size change, a python script is used. Python was chosen for its quality of image recognition and data analysis libraries. For image analysis, OpenCV2 was used for the vast amount of algorithms to choose from. Pandas was used for data analysis for ease of use and compatibility with plotting libraries. Lastly, a GUI was made for easier user interactions with Tkinter. While clunky, Tkinter is easy to program within the tighter timeframe of this coding project. The algorithms in OpenCV2 are used for all image analysis and no processing is done through a Neural Network. While convolutional neural networks are very powerful image analysis and segmenting tools, with the limited time in this project, other tools were deemed more viable.

Code:

The program is split across 5 main scripts. Each script is used to house a different function of the program.

The main image processing occurs in the script “individualImageSegmenter.py”. This is where the image is segmented and dimensions are captured. It is the most important part of the program, with all other parts wrapping around to make user experience easier. The image analysis process starts with taking in an image, either it being a silhouette of the particle or the phosphor image. It is then converted into a true black and white image, with the particle being white to aid in contrast. From there, an open morphology¹ is used to remove the thinner wires from the image. With the now cleaned contour, a user-defined amount is taken off the top to

account for the screw that is used to hold the particle that cannot be removed through the morphology. From the contour and user defined shape, either an eclipse or rectangle is fitted around the object. The fitting function returns a length and width that are used as the dimensions of the object. The outline of the particle is also drawn and returned.

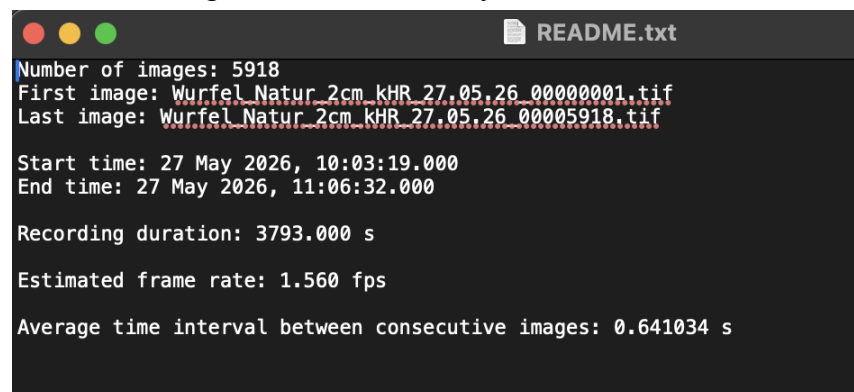
The script “folderSegmenter.py” houses the logic to pull out the data needed from the input folder and along with “timeStamp.py”, construct the necessary output CSV files. It loops through designated folders, pulls out meta data stored in excel and text files to gather time stamp information, and passes through each image to the segmenter script. When the dimensions and output image are returned, the dimensions are added to a data frame and the image is passed through to be updated in the GUI. Each camera is handled separately, and corresponding data is then merged into one CSV by matching the nearest time stamps.

The script “GUI.py” contains code for the design of the GUI and allows for updates to the images shown. This includes user input gathering

Lastly, “GraphMaker.py” is used to create output graphs of relevant data. This is done through a button in the GUI.

Data Format:

Data for this program has to be in a specific format. There should be one main folder housing two subfolders. Naming of these folders does not matter. The silhouette camera images should be in a folder containing a .txt file that strictly is the format of the following image image:



```
Number of images: 5918
First image: Wurfel_Natur_2cm_kHR_27.05.26_00000001.tif
Last image: Wurfel_Natur_2cm_kHR_27.05.26_00005918.tif

Start time: 27 May 2026, 10:03:19.000
End time: 27 May 2026, 11:06:32.000

Recording duration: 3793.000 s
Estimated frame rate: 1.560 fps
Average time interval between consecutive images: 0.641034 s
```

The phosphor images should be in a separate folder containing the images and an excel file that has the time stamps. The general format is the following:

Date	Start Measure/ No	Time [Timestamp]	Time [hh:mm:ss]	Time [s]	
5/27/26	9:52:01	0	10:03:46 AM	12:11:45 AM	705
5/27/26	9:52:01	1	10:10:00 AM	12:17:59 AM	1079
5/27/26	9:52:01	2	10:10:25 AM	12:18:24 AM	1104
5/27/26	9:52:01	3	10:12:47 AM	12:20:46 AM	1246
5/27/26	9:52:01	4	10:15:56 AM	12:23:55 AM	1435
5/27/26	9:52:01	5	10:17:33 AM	12:25:32 AM	1532
5/27/26	9:52:01	6	10:18:01 AM	12:26:00 AM	1560
5/27/26	9:52:01	7	10:19:31 AM	12:27:30 AM	1650
5/27/26	9:52:01	8	10:20:20 AM	12:28:19 AM	1699
5/27/26	9:52:01	9	10:21:34 AM	12:29:33 AM	1773
5/27/26	9:52:01	10	10:23:45 AM	12:31:44 AM	1904
5/27/26	9:52:01	11	10:24:01 AM	12:32:00 AM	1920
5/27/26	9:52:01	12	10:25:28	12:33:27 AM	2007
5/27/26	9:52:01	13	10:25:59 AM	12:33:58 AM	2038
5/27/26	9:52:01	14	10:26:49 AM	12:34:48 AM	2088
5/27/26	9:52:01	15	10:27:30 AM	12:35:29 AM	2129
5/27/26	9:52:01	16	10:28:11 AM	12:36:10 AM	2170
5/27/26	9:52:01	17	10:28:44 AM	12:36:43 AM	2203
5/27/26	9:52:01	18	10:29:14 AM	12:37:13 AM	2233
5/27/26	9:52:01	19	10:29:51 AM	12:37:50 AM	2270
5/27/26	9:52:01	20	10:30:39 AM	12:38:38 AM	2318
5/27/26	9:52:01	21	10:31:12 AM	12:39:11 AM	2351
5/27/26	9:52:01	22	10:31:38 AM	12:39:37 AM	2377

The following is a picture of the over file structure. Along with this, the github repository contains a sample folder as well.

trailRun	Jul 23, 2026 at 12:57 PM	--	Folder
Cubes_chr_phosphorthermometry_27.05.26 copy	Jul 23, 2026 at 1:11 PM	--	Folder
Cube_natural_2cm_kHR_27.05.26.xlsx	Jul 23, 2026 at 12:59 PM	11 KB	Micros...k (.xlsx)
frame_00008_0.png	Jul 8, 2026 at 11:01 PM	5 KB	PNG image
frame_00008_1.png	Jul 8, 2026 at 11:01 PM	4 KB	PNG image
Wurfel_Natur_2cm_kHR_27.05.26	Today at 11:03 PM	--	Folder
README.txt	Jul 23, 2026 at 12:57 PM	344 bytes	Plain Text
Wurfel_Natur_2cm_kHR_27.05.26_00000001.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000002.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000003.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000004.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000005.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000006.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000007.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000008.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000009.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image
Wurfel_Natur_2cm_kHR_27.05.26_00000010.tif	May 27, 2026 at 8:03 AM	2.8 MB	TIFF image

Instructions:

The following is an image of the GUI that will be referenced in the following section.

There are six input variables they are in order: Output File Path, Shape, Chop Amount, Main Dimension Length, Frame Skip Number, and Input Path.

There are 3 methods of input. The first is with a text box where the user types in their input and clicks the button next to it to confirm the selection. If the background of the button turns green, then the input and been registered

Second is a drop down menu. The default value is Circle and can be switched by clicking the proper option. No additional button must be pressed.

The third is a file browser. Once the button is clicked, a file browser appears and prompts the user. The user should select the data folder that they would like to process. This should be the

parent folder holding the two subfolders of silhouette images and phosphor images. You should not select one of these sub folders, otherwise the program will fail to start.

Output File Path: The name of the CSV files that you wish to output

Shape: Shape of the particle that you wish to examine

Chop Amount: Decimal of the amount that you wish to chop off. Should range from 0.0-1.0. Recommended and default value at 0.12. Though fine tuning based on in progress image should be used

Main Dimension Length: Length of main dimension. This program assumes that a particle is either a square prism or sphere to start.

Frame Skip Number: The program will look at every i th frame, with i being the frame skip number. (Ex: FSN = 5, the program will look at the 0th, 5th, 10th, 15th, etc) Used to speed up processing of large batches of files. Only modifies the silhouette images processing as the phosphor camera is assumed to have much lower picture count. This has a default value of 1 (Every frame will be checked). Any values set to below 1 will be defaulted to 1. It is not recommended to go above 50 for graphing purposes since large changes in volume happen quickly.

Input Path: Path to the data folder to be processed. This should be the parent folder holding the two subfolders of silhouette images and phosphor images. You should not select one of these sub folders, otherwise the program will fail to start.

Once all values are imputed that are necessary, one can click the large green start button. At that point, the images will start to be processed. Once the process is completed, the output CSV's are created and the start button will be able to be clicked again. The graph button can be clicked to use the current path of the output csv to generate a premade graph template. If there are any error messages that appear in the terminal, the program will most likely need to be closed out of and rerun.

To run the program, run the "[gui.py](#)" script. This will load all necessary other files. If only graphing is needed, one can run the "[graphingMaker.py](#)" script. This will pull up a file picker to choose the appropriate .csv file for graphing. A terminal input will also be needed to specify the shape for volume calculations.

Improvements:

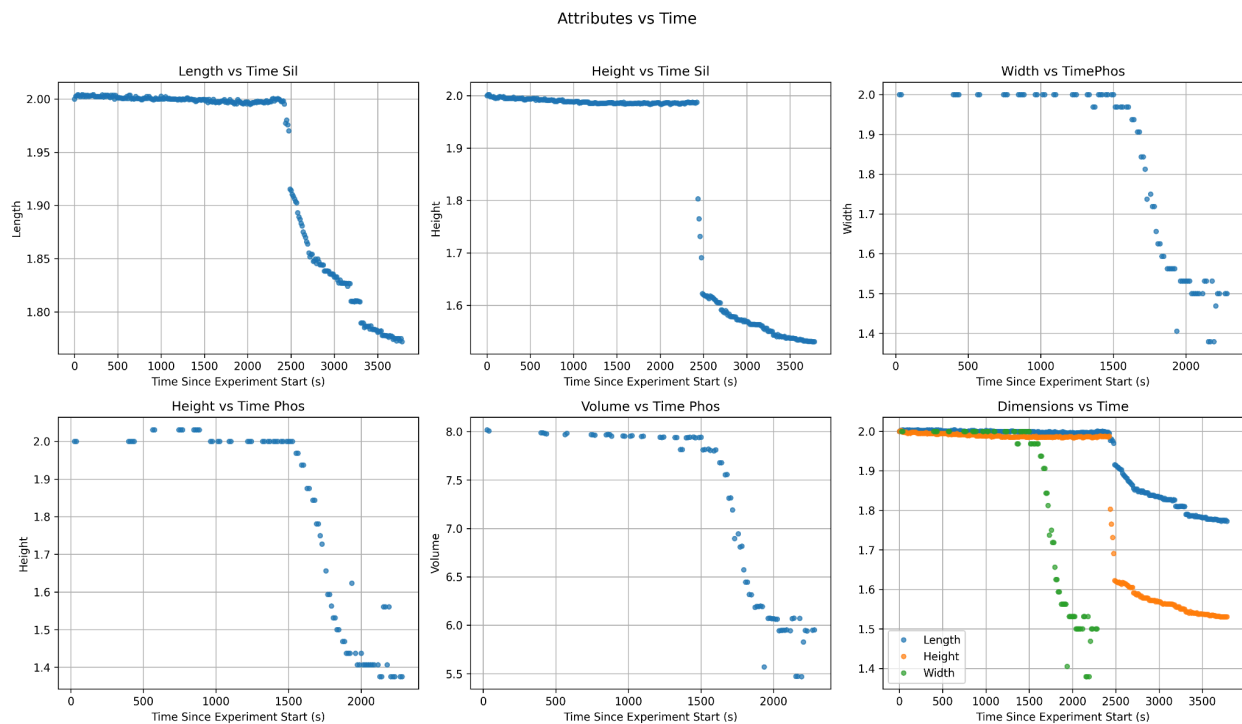
Due to the limited time available to complete this project, there are certain functionalities that were not able to be added. The following is a list of recommended functions to add to the program:

1. Proper input screening
2. Ability to handle non-uniform objects
3. Adding a way to change chop amount to prevent percentage based chopping from being affected by shrinking
4. Use CustomTkinter for better looking GUI
5. Toggles to allow individual inputs for timestamps and data folders

Conclusion:

All in all, the program and methodology outline works as designed and returns output. This is easily graphable to show correlations. This will be used further by Dr. Andrea Dernbecher and Fabienne Ryyle in their experimentation with wood pyrolysis.

Below are the graphs generated for example dataset:



Sources:

Github for project - <https://github.com/iaz228/Research-on-wood-pyrolysis>

Filehosting for example silhouette images -

<https://tu-dortmund.sciebo.de/s/LLSptx86SwP4Kse?dir=/>

Filehosting for example Phosphor images -

https://drive.google.com/drive/folders/1_lbLZrY_OffUZDj2BqFX6fFNTNhlxOke?usp=sharing

1 - <https://www.geeksforgeeks.org/python/python-opencv-morphological-operations/>